SW Engineering CSC648/848 Fall 2025
Milestone4 GatorTutors
Team 14

Name	Role
William Wong	Team Lead
James Barr	Frontend Lead
Ethan Niu	Frontend Engineer
Uchenna Duru	Backend Lead
Randy Chen	Backend Engineer
Cyrus Molina-Perez	Github Maintainer

Team lead email: wwong23@sfsu.edu

December 19, 2025

Table of Contents

[Title Page](#)
[Table of Content](#)
[Product Summary](#)
[Usability test plan](#)
[QA Test Plan & QA Testing](#)
[Peer Code Review](#)
[Self Check on Best Practices for Security](#)
[Self Check of the Adherence to Original Non-Functional Specs](#)
[Use of GenAI](#)

Product Summary

Product Name: GatorTutors

GatorTutors is a tutoring platform built exclusively for San Francisco State University students, connecting learners with verified peer tutors through a secure web interface that streamlines booking, communication, and session tracking. Students can search for tutors by course number, subject, or availability to instantly book on-line or in-person sessions while tutors can manage schedules and review their sessions with ease. Unlike generic services, our product integrates directly with SFSU's course catalog, ensuring matches at the class-code level so students can receive guidance from peers who have already taken the same classes and understand professor expectations. By combining flexibility and familiarity, GatorTutors empowers students to learn from one another, expands academic support beyond traditional tutoring centers, and creates opportunities for peer mentorship.

Major Functions:

- Tutoring site works exclusively for SFSU students
- Students can create an account using their student email
- Students can login to their account
- Students can search for tutors using optional filters
- Students can book tutoring sessions with an available or specific tutor
- Students can communicate with tutors through messages
- Students can view booked tutoring sessions in their dashboard
- Students can view their inbox in their dashboard
- Students can view their profile in their dashboard
- Students can apply to become a tutor
- Tutors can login to their account
- Tutors can view booked sessions in their dashboard
- Tutors can set their availability in their dashboard

URL: http://18.219.122.227

Usability test plan

1. **Test objectives**
   The primary functionality we are testing is whether students can successfully book a session with an available tutor and view that session on their dashboard. This feature is critical because it represents the core purpose of the platform. By verifying that the booking process works correctly and quickly, we ensure that students can rely on the system sessions without confusion or errors. Testing the dashboard display is equally important as it provides students with a clear record of their upcoming commitments and helps them manage their time effectively.

2. **Test background and setup**
   a. System setup, starting point, what HW tester needs to have: Users can browse available tutors without needing an account, but booking a session requires logging in or signing up with their credentials. After logging in, students can search for a tutor and simply click the “Book Session” button on a tutors session. Once the booking is comfirmed, the platform will automatically redirect users to their dashboard where they will be able to view all booked sessions in one place.
   b. Who are the intended users: The intended users are exclusively SFSU students and peer tutors verified through their official SFSU emails. Students can search for tutors based on course criteria while peer tutors can manage their availability. By restricting access to the campus community, the platform ensures a safe, trusted environment.
   c. URL of the system to be tested: http://18.219.122.227/search.html
   d. Test environment: The test can be conducted anywhere as long as a user has access to the internet. No cameras will be in use to capture user interactions and there will be no monitoring. Testers need no prior training to complete the booking sessions and the only constraint is that they have a registered account.

3. **Usability Task description**
   a. Open the GatorTutors website in a browser.

b. If you do not already have an account, sign up using your SFSU email. If you already have one, login.
c. Use the search bar to locate an available tutor.
d. On the tutors available session, click the “Book Session” button to schedule a session.
e. Confirm that the system redirects you to your dashboard.
f. Verify that the booked session appears with correct details.

4. Plan for Evaluation of effectiveness

a. Define clear success criteria (session booked and visible on dashboard)
b. Recruit representative SFSU students to perform the booking task
c. Track task completion rates to see how many succeed without errors
d. Not any user errors, confusion, or repeated attempts
e. Collect user feedback on ease of use and satisfaction
f. Compare results against expected outcomes to identify gaps

5. Plant for Evaluation of efficiency

a. Record the time it takes for users to complete the booking and dashboard viewing task
b. Count the number of clicks or steps required to finish the process
c. Track whether users can complete the task on their first attempt
d. Note any uneccessary actions or detours taken during the task
e. Compare average completion time across multiple participants
f. Identify points where users hesitate or slow down
g. Evaluate if system responses occur quickly
h. Assess whether the dashboard updates instantly after booking

6. Plan for Evaluation of user satisfaction

Please take a moment to answer the following questions.	**Strongly Agree**	**Agree**	**Neutral**	**Disagree**	**Strongly Disagree**
1. It was easy for me to book a tutoring session and view it on my dashboard	○	○	○	○	○
2. The interface was clear and I understood what to do at each step	○	○	○	○	○

3. I am satisfied with the overall experience of using the platform to book and track sessions	○	○	○	○	○

7. **GenAI use**
    a. Microsoft Copilot, GPT-5
    ChatGPT, GPT-5.2
    Google Gemini, Gemini 2.0
    b. GenAI was leveraged to support the usability test plan development process in two ways. First, it was used as an ideation tool to help generate and refine tests that could be used on the product, helping the team explore a wider range of options. Second, it assisted in debugging and improving code by providing explanations and optimizations when the testing failed.
    c. For example, the team needed to design usability tests for the student dashboard. The following prompt was used to come up with ideas to how it should be tested: “Suggest usability test cases for a student dashboard on a tutoring website that includes upcoming sessions, an inbox, and student profile.”. Another example involved debugging code to display the user's profile even when the database fields were empty instead of returning an error. We used this prompt to explain the error and suggest a solution: “Explain why this SQL update is failing when no fields are passed and suggest a fix to this code:...”
    d. Microsoft Copilot - HIGH
    Google Gemini - LOW

QA Test Plan & QA Testing

1.**Test Objectives**

The objective of this QA test plan is to verify that students can successfully book sessions with tutors, with the session appearing on their dashboard, and the system updates components correctly.

2. **HW and SW setup:**

Hardware Setup:

- Laptop or desktop computer with a standard keyboard and mouse/trackpad

Software Setup:

- Operating system: Windows 10/ Windows 11
- Web Browsers Used for Testing:
  - Google Chrome
  - Mozilla Firefox:
- Stable internet connection

Application URL:

- http://18.219.122.227

3. **Feature to be tested.**

The features under test for the booking session include: Searching for available tutors, booking a session through the “Book Session” button, automatic redirection to the student dashboard, display of the booked session with correct details, and removal of the session from available sessions list.

4. **QA test plan: in table format:**

Test #	Test title	Test Description	Test input	Expected output	Test results
1	Search for available tutor	Verify that the search bar returns available tutors	Enter a course code (e.g. MATH).	A list of available tutors with at least one session visible.	
2	Book a tutoring session	Test that the student can successfully book	Click “Book Session” on the tutors	A pop up message should confirm the session was booked	

		a session	session	and the page should redirect to user dashboard	
3	Verify the student dashboard	Ensure the booked session appears on the dashboard	After booking, view the student dashboard	The booked session should appear with the correct information	

5. Testing on two major platforms:
   - Google Chrome - PASS
   - Firefox - PASS

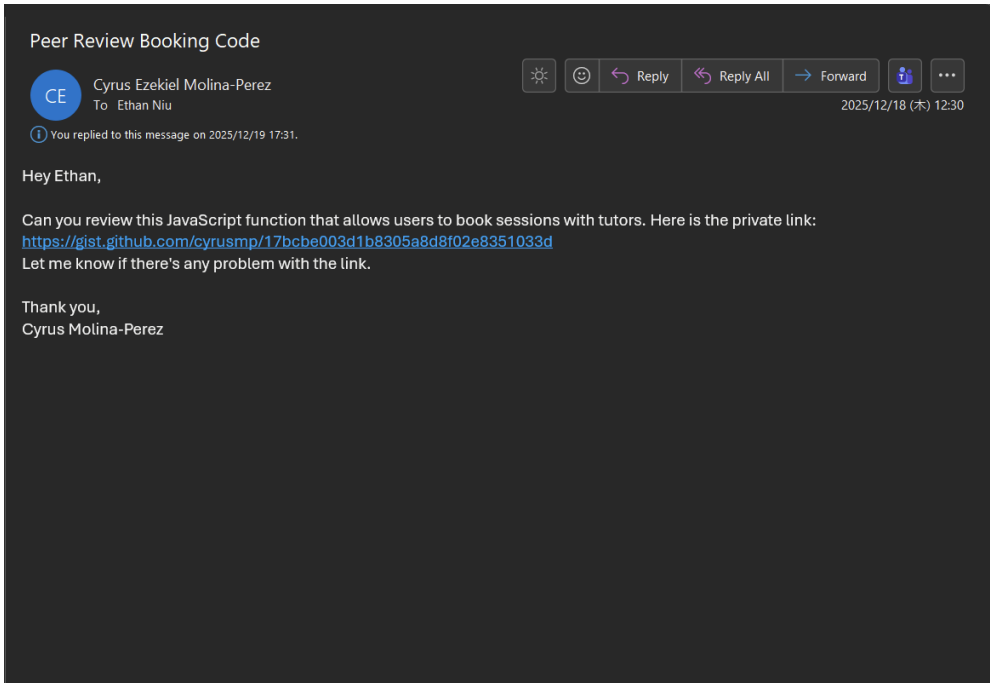
6. GenAI use:
   - **What GenAI tool and version:**
     ChatGPT model GPT-5.2

   - **Explain briefly how you used the tool and what benefit it offered.**
     It provided a clear explanation of what the key functional differences are between QA Testing and the usability test plan. ChatGPT was used to give an example of what a test plan should look like given the Milestone 4 requirements.

   - **Provide brief examples of key examples and prompts**
     ChatGPT was given the milestone 4 requirements and was asked if it could help with creating a QA test plan, the given example was used as a base for the QA test plan, and modified to fit our website.

   - **RANK utility of GenAI**
     GenAI utility: MEDIUM

Peer Code Review



Peer Review Booking Code

CE Cyrus Ezekiel Molina-Perez
To Ethan Niu

Reply | Reply All | Forward

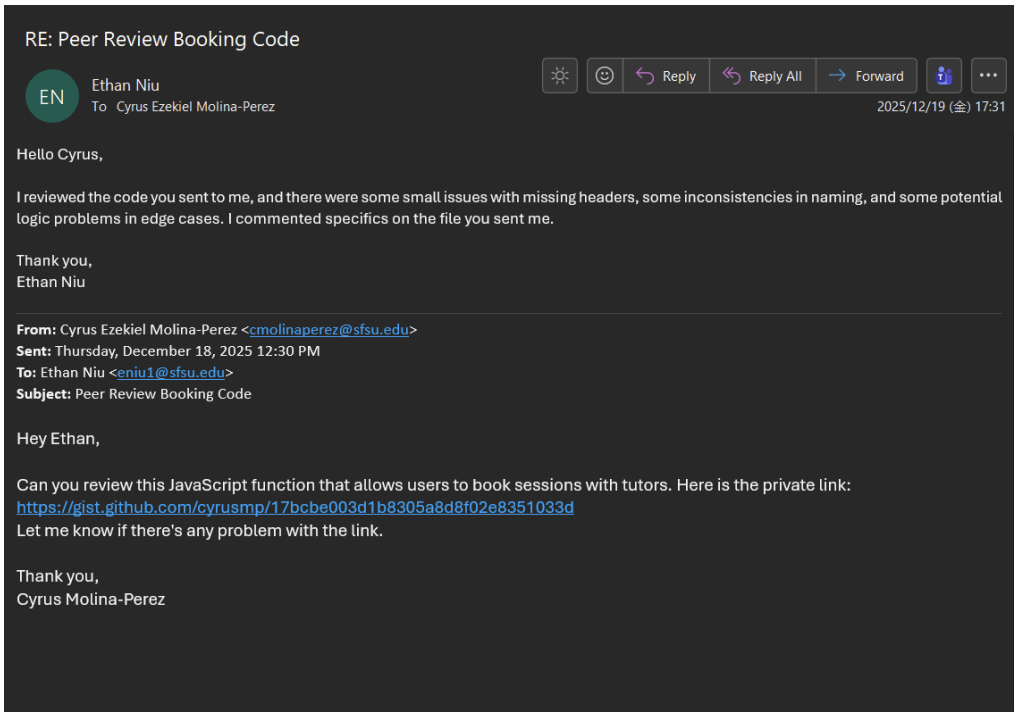
2025/12/18 (木) 12:30

You replied to this message on 2025/12/19 17:31.

Hey Ethan,

Can you review this JavaScript function that allows users to book sessions with tutors. Here is the private link:
https://gist.github.com/cyrusmp/17bcbe003d1b8305a8d8f02e8351033d
Let me know if there's any problem with the link.

Thank you,
Cyrus Molina-Perez



RE: Peer Review Booking Code

EN Ethan Niu
To Cyrus Ezekiel Molina-Perez

Reply | Reply All | Forward

2025/12/19 (金) 17:31

Hello Cyrus,

I reviewed the code you sent to me, and there were some small issues with missing headers, some inconsistencies in naming, and some potential logic problems in edge cases. I commented specifics on the file you sent me.

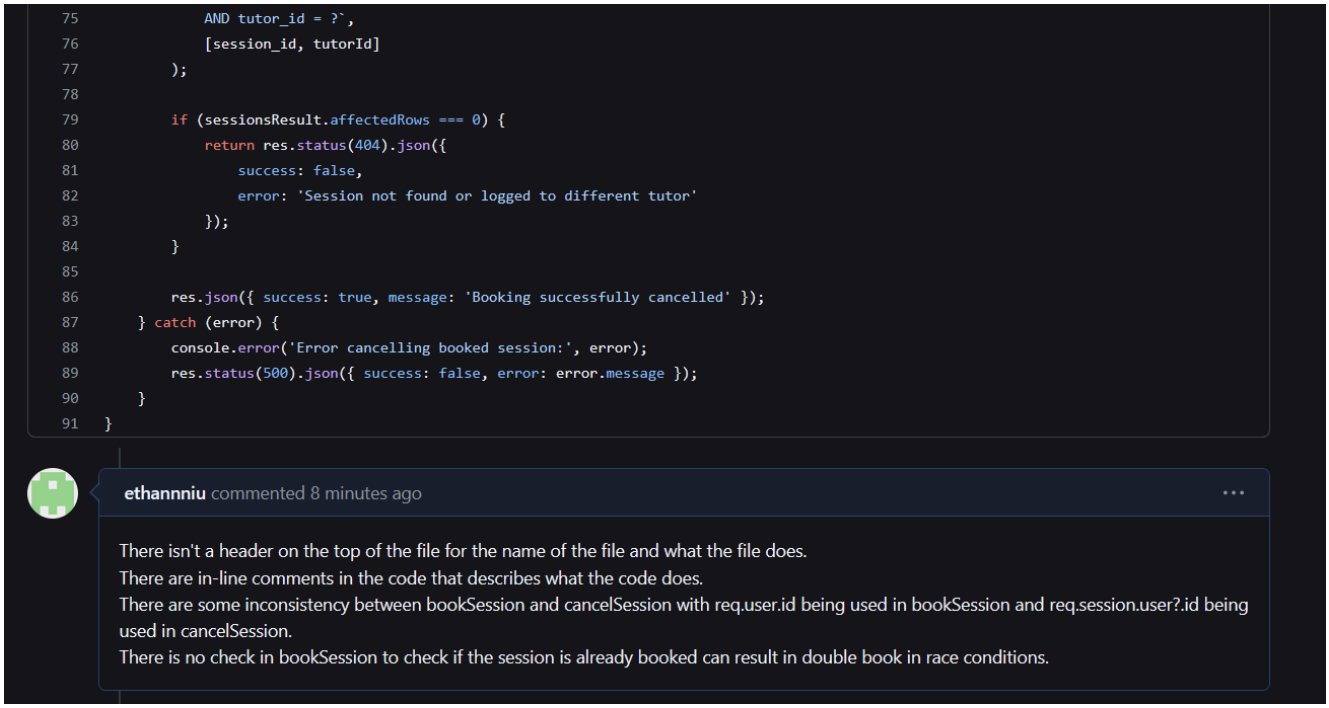
Thank you,
Ethan Niu

From: Cyrus Ezekiel Molina-Perez <cmolinaperez@sfsu.edu>
Sent: Thursday, December 18, 2025 12:30 PM
To: Ethan Niu <eniu1@sfsu.edu>
Subject: Peer Review Booking Code

Hey Ethan,

Can you review this JavaScript function that allows users to book sessions with tutors. Here is the private link:
https://gist.github.com/cyrusmp/17bcbe003d1b8305a8d8f02e8351033d
Let me know if there's any problem with the link.

Thank you,
Cyrus Molina-Perez



GenAI session for code review:

Sure — this is a solid start and the intent is clear. I'll review it in a **backend / security / correctness** lens and then suggest **concrete improvements** with examples.

High-level assessment ✅

What you did well

- Clear separation of concerns (`bookSession` vs `cancelSession`)
- Parameterized queries (✅ protects against SQL injection)
- Proper HTTP status codes in most cases
- Good comments explaining assumptions (auth middleware, etc.)
- Updating `tutoring_sessions.is_booked` consistently

Key issues & improvements 🔴🟡🟢

🔴 1. Inconsistent auth source (`req.user` vs `req.session.user`)

You use **two different auth mechanisms**:

```
// bookSession
const student_id = req.user.id;

// cancelSession
const tutorId = req.session.user?.id;
```

⚠️ This is dangerous and confusing.

Why it matters

- If your app standardizes on JWT / authMiddleware → `req.user`
- Mixing session-based and token-based auth can break access control

✅ Recommendation
Use **one** auth source everywhere (likely `req.user`):

```
const tutorId = req.user.id;
```

🔴 2. No transaction (data integrity risk)

Your booking flow is **not atomic**:

```
INSERT INTO bookings ...
UPDATE tutoring_sessions SET is_booked = 1
```

If the second query fails:

- Booking exists

- Session still appears available

🚨 Same problem in `cancelSession`.

✅ **Fix: use a transaction**

```
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();

  const [result] = await conn.query(
    `INSERT INTO bookings (student_id, tutor_id, session_id)
     VALUES (?, ?, ?)`,
    [student_id, tutor_id, session_id]
  );

  await conn.query(
    `UPDATE tutoring_sessions
     SET is_booked = 1
     WHERE id = ? AND is_booked = 0`,
    [session_id]
  );

  await conn.commit();
  res.json({ success: true, booking_id: result.insertId });
} catch (err) {
  await conn.rollback();
  throw err;
} finally {
  conn.release();
}
```

🔴 **3. Race condition: double booking possible**

Two students can book the same session at the same time.

Current logic:

- No check that `is_booked = 0`
- No unique constraint enforcement

✅ Best practice

Update with condition:

```
UPDATE tutoring_sessions
SET is_booked = 1
WHERE id = ? AND is_booked = 0
```

Then verify:

```
if (updateResult.affectedRows === 0) {
  return res.status(409).json({
    success: false,
    error: "Session already booked"
  });
}
```

🟢 Bonus: add DB-level protection

`UNIQUE (session_id)`

on `bookings`

🟡 4. Missing role validation

You assume:

- `bookSession` → student
- `cancelSession` → tutor

But you never enforce role explicitly.

✅ Add role checks

```
if (req.user.role !== 'student') {
  return res.status(403).json({ error: "Students only" });
}

if (req.user.role !== 'tutor') {
```

```
  return res.status(403).json({ error: "Tutors only" });
}
```

🟡 **5. `session_id` trust issue**

In `cancelSession`, the client supplies both:

```
{ booking_id, session_id }
```

⚠️ A malicious client could mismatch them.

✅ **Safer approach**
Derive `session_id` from the booking:

```
DELETE FROM bookings
WHERE booking_id = ? AND tutor_id = ?
RETURNING session_id
```

Then update the returned session.

🟡 **6. HTTP semantics**

You say:

```
// DELETE /api/book
```

But you still read from `req.body`.

🟡 This is allowed, but REST-wise it's better to:

```
DELETE /api/book/:booking_id
```

And use:

```
req.params.booking_id
```

🟢 **7. Error exposure**

```
res.status(500).json({ error: err.message });
```

In production, this can leak DB details.

✅ Better:

```
res.status(500).json({
  success: false,
  error: "Internal server error"
});
```

Summary of recommended fixes

Area	Fix
Auth	Use `req.user` consistently
Data integrity	Wrap booking/cancel in transactions
Race conditions	Conditional updates (`is_booked = 0`)
Authorization	Enforce roles (student/tutor)
Security	Don't trust client `session_id`
API design	Improve DELETE route semantics
Errors	Hide internal error messages

Overall verdict ⭐

Functionality: ✅ Good
Security: ⚠️ Needs tightening

Concurrency safety: ❌ Needs transactions
Design clarity: 🟡 Minor inconsistencies

If you want, I can:

- Refactor this into a **production-ready version**
- Add **unit tests**
- Review your **DB schema**
- Align this with **JWT auth middleware**

Just tell me what you want next 👍

What GenAI tool and version you used:

ChatGPT version GPT-5.2

Explain briefly how you used the tool and what benefit it offered.

I asked ChatGPT to help me review the code, and it offered a comprehensive rundown of problems in the code that might have otherwise not have been made known.

Provide brief examples of key examples and prompts.

I pasted the code in the file to ChatGPT and asked “can you review this code?”

RANK utility of GenAI here as LOW or MEDIUM or HIGH

Medium

Self Check on Best Practices for Security

Our team conducted a security review of the tutoring platform to ensure all major assets are protected from any threats. The table below summarizes the key assets, the threat they face, and solutions we've implemented. We confirm that all user passwords stored in the database are encrypted and that our system performs input validation on all user-submitted fields. We also validated that the search bar input is limited to 40 alphanumeric characters, registered emails must end in "@sfsu.edu", and that users explicitly accepted the Terms & Conditions before account creation.

Asset to be protected	**Types of possible/expected attacks**	**Consequence of security breach**	**Strategy to mitigate/protect asset**
Server	Brute-force SSH login attempts, stolen SSH private keys, unauthorized remote access, privilege escalation attacks	Full server takeover, insertion of malicious code, loss of system integrity	Use SSH key-based authentication
Database	SQL injection, credential theft, data scraping, privilege misuse	Exposure of sensitive academic information, tampering with session availability, loss of user trust	Enforce strong DB passwords, restrict DB access
User Accounts	Credential stuffing, phishing attempts, session hijacking via stolen cookies	Unauthorized booking or cancellation of sessions, access to private messages, reputational damage	Require strong passwords

Self Check of the Adherence to Original Non-Functional Specs

The system is accessible via modern web browsers — **DONE**.
The application has been tested and is accessible using modern browsers including Chrome, Firefox, and Edge on desktop systems.

The system is deployed on a publicly accessible server — **DONE**.
The application is deployed on a publicly accessible AWS EC2 instance using a public IP address and can be accessed by instructors for grading and testing.

The system supports multiple concurrent users — **DONE**.
The backend supports multiple user sessions simultaneously and has been exercised by multiple team members without conflicts or failures.

The system provides reasonable response times for user actions — **DONE**.
Page loads, searches, messaging, and dashboard interactions complete within acceptable response times under normal usage conditions.

The system securely stores user authentication credentials — **DONE**.
User passwords are securely hashed and stored in the database following standard security practices.

The system validates user input to prevent malformed or unsafe data — **DONE**.
Input validation is enforced for critical fields including search inputs, registration data, email format, and required form fields.

The system restricts access to authenticated users for protected features — **DONE**.
Sensitive features such as dashboards, messaging, and booking are accessible only to authenticated users.

The system enforces SFSU-specific access constraints — **DONE**.
User registration is restricted to valid SFSU email addresses, ensuring the platform remains exclusive to the SFSU community.

The system provides fault tolerance and basic error handling — **DONE**.
User-facing error messages are displayed for common failure scenarios, and the system recovers gracefully from invalid actions.

The system maintains data consistency across user sessions — **DONE**.
User profiles, messages, bookings, and session data remain consistent across sessions and page reloads.

The system provides a usable and intuitive user interface — **DONE**.
The interface follows consistent navigation patterns and was refined based on usability feedback collected during earlier milestones.

The system separates frontend and backend responsibilities — **DONE**.
Frontend presentation and backend logic are clearly separated through defined APIs and modular code organization.

The system supports maintainable code organization — **DONE**.
Code is structured into logical components with consistent naming conventions to support readability and maintenance.

The system supports future feature extensibility — **DONE**.
The application architecture allows for additional features and enhancements to be added without major refactoring.

The system protects sensitive user data — **DONE**.
Sensitive information such as passwords and verification data is never exposed in client-side code or logs.

The system provides monitoring and usage visibility — **DONE**.
Google Analytics is integrated to provide visibility into site usage and user engagement.

The system remains available during scheduled demo and grading periods — **DONE**.
The deployed application has remained stable and accessible throughout final testing and demo preparation.

The system complies with all course-defined deployment and documentation requirements — **DONE**.

Use of GenAI

GenAI tools were used to support planning, review, and documentation tasks throughout the project rather than to replace core implementation work. ChatGPT was used to help draft and review usability and QA test plans, refine test case coverage, and improve clarity and consistency in project documentation.

GenAI was also used as a supplementary reviewer during peer code review to identify potential issues related to readability, structure, and best practices. All GenAI suggestions were evaluated by team members, and only accurate and relevant feedback was incorporated.